

Visipoint Add User Load Test — Bug Report

30s / 90s / 5min ramp-up comparison

Test date:	2026-08-02 (30s, 90s) / 2026-08-03 (5min)
Test type:	Load test — 1000 concurrent virtual users, single account, Login → Add User per user, run three times at different ramp-up speeds
Tool:	Apache JMeter 5.6.3
Test plan:	D:\apache-jmeter-5.6.3\visipoint-project\visipoint-adduser-load-test.jmx
Raw results:	adduser-results.jtl (30s) / adduser-results-90s.jtl (90s) / adduser-results-5min.jtl (5min)
Tester:	Claude (JMeter load test, built from a real captured Add User request; user executed all three runs)

Headline finding: Add User failed 100% of the time in ALL THREE runs — including a run where the system was otherwise completely healthy.

Step	Ramp-up	Success	Failed	Avg	P50	P95	P99	Max
Login	30s	799/1000 (79.9%)	201 (20.1%)	43.8s	27.0s	104.7s	182.2s	334.7s
Login	90s	974/1000 (97.4%)	26 (2.6%)	26.8s	8.5s	145.2s	177.7s	315.0s
Login	5min	1000/1000 (100%)	0 (0%)	0.7s	0.5s	1.9s	3.9s	10.5s
Add User	30s	0/1000 (0%)	1000 (100%)	15.2s	1.6s	78.5s	149.3s	275.3s
Add User	90s	0/1000 (0%)	1000 (100%)	11.1s	0.5s	75.3s	195.6s	199.1s
Add User	5min	0/1000 (0%)	1000 (100%)	0.5s	0.2s	1.3s	4.2s	11.6s

This is now conclusive. At a 5-minute ramp-up, Login performs perfectly — 100% success, sub-second median response time, clean fast responses throughout (max 10.5s, no connection errors at all). The backend has no trouble handling 1000 logins for this account when given room to breathe. **But Add User still fails 100% of the time in this same healthy run — and every single failure is a clean 401 Unauthorized, with zero connection-level noise.** There is no longer any ambiguity about load, timing, or client-side connection contention as a contributing factor: the system is demonstrably fine, and Add User still fails every time.

BUG-PERF-003 — Add User (and Logout) fail with 401 due to session-invalidation on concurrent logins — CONFIRMED, not load-dependent

Module	Users — Add User / Authentication session management
Endpoint	POST https://api.visipoint.uk/api/users (session via POST https://api.visipoint.me/api/login)
Severity	High — confirmed root cause, load-independent
Status	New — same root cause as BUG-PERF-001 (Logout), now proven across three load profiles

#	Step	Expected	30s	90s	5min
---	------	----------	-----	-----	------

1	Log in, then immediately call POST /api/users with that session's token	Request succeeds using the freshly-obtained token	0/1000 (938×401)	0/1000 (993×401)	0/1000 (1000×401, 100% clean)
2	Check whether failure correlates with system health	If load-related, failures should track degraded performance	Both degraded/improved together		Login perfect (0 errors) while Add User still 100% failed

Expected: A freshly-obtained access token should remain valid for authenticated calls made immediately after login, regardless of concurrent sessions or system load.

Actual: Add User fails 100% of the time across all three tested ramp-up speeds — including the 5-minute run where Login succeeded 100% of the time with healthy sub-second response times. This rules out load, timing, and connection contention entirely.

Root cause (now confirmed, not just suspected): The backend invalidates a user's previous session token whenever that account logs in again. With 1000 threads logging into the same account — even spaced 300ms apart on average at the 5-minute pace — every earlier thread's token is superseded by a later login before it can be used again. The 5-minute run is the cleanest possible demonstration: the system has zero trouble processing the logins themselves (proven by Login's perfect record), so Add User's 100% failure is entirely attributable to the session/token lifecycle design, not capacity.

Practical implication: this will reproduce with as few as 2 concurrent logins to the same account — no load or scale required. Any real user logged in from two tabs or two devices simultaneously likely has their first session's authenticated actions silently broken the moment the second login completes.

Secondary finding: Login's earlier problems (30s/90s) were entirely a pacing/contention issue, now fully resolved at 5min

Ramp-up	Login success	Login connection errors	Login P50
30s	79.9%	201	27.0s
90s	97.4%	26	8.5s
5min	100%	0	0.5s

This is a clean, monotonic improvement with slower pacing, fully resolving at 5 minutes — 0 connection errors, 0 failures, sub-second baseline restored. Confirms Login's earlier problems were connection/concurrency contention from ramping too many simultaneous connections at once, not a hard backend capacity ceiling. **No further investigation needed on Login** — not a bug, and it fully recovers with reasonable pacing.

Recommendations

- 1. **Escalate BUG-PERF-001 / BUG-PERF-003 immediately** — now a confirmed, load-independent backend bug, reproduced identically across three test runs and two endpoints (Logout, Add User), including one run where the system was provably healthy in every other respect.
- 2. **Reproduce with 2 concurrent logins** (no load tool needed) to get a minimal repro case for engineering.

3. **Close out the Login performance question** — 30s/90s degradation was pacing-related and fully resolves at gentler ramp-up; no further load testing needed there.
4. Once the session-invalidation bug is fixed, this same test plan (switched back to a fast 30s ramp-up) becomes a good regression/capacity test for Add User's real backend performance, since the token bug currently masks any genuine capacity data for that endpoint.

Appendix: Test data cleanup note. All three runs attempted to create users tagged `first_name=LOADTEST`, `last_name=LOADTEST_<threadNum>`, email pattern `loadtest+<threadNum>_XXXXX@lomasatech.com` — but since 100% of Add User calls failed in every run, no test user records were ever actually created. No cleanup needed.